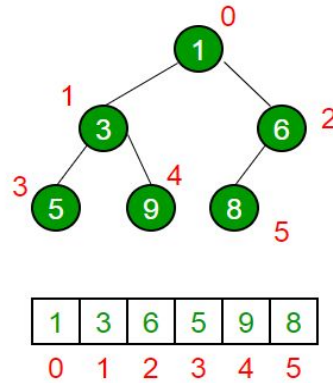


Heap



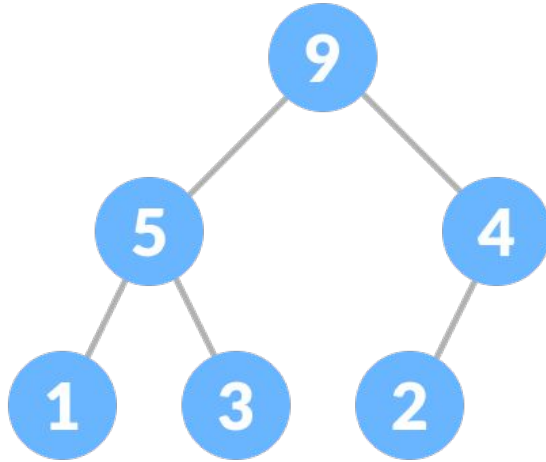
Introduction to Heap



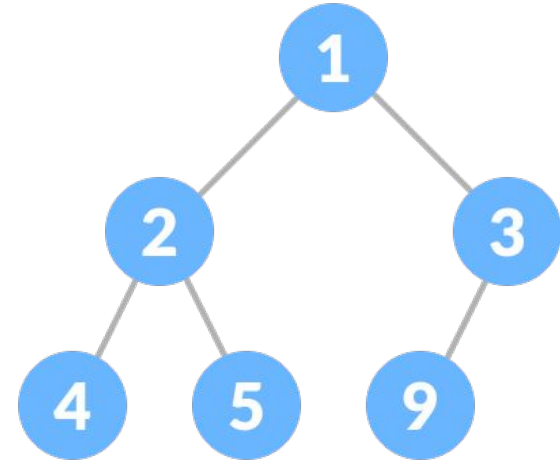
Types of Heap



Generally there are two types of heaps



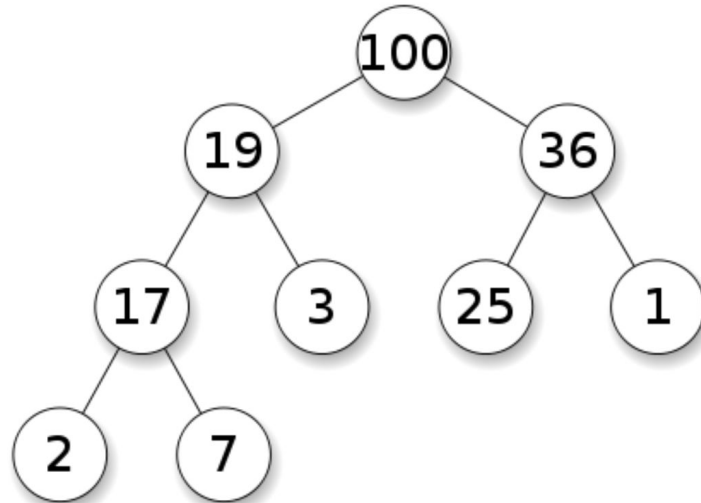
- Max-Heap



- Min-Heap

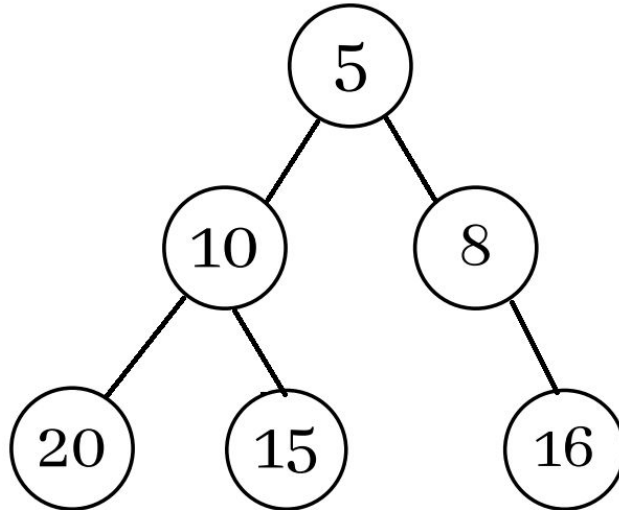
Max-Heap

- The value present at the root node must be greatest among the values present at all of its children. The same property must be recursively true for all sub-trees in that Binary Tree.



Min-Heap

- The value present at the root node must be minimum among the values present at all of its children. The same property must be recursively true for all sub-trees in that Binary Tree.



Contd...

- Since a Heap is a special Tree-based data structure in which the tree is a complete binary tree, the above types of data structure are also called a **binary heap**.

Some other types of Heap:

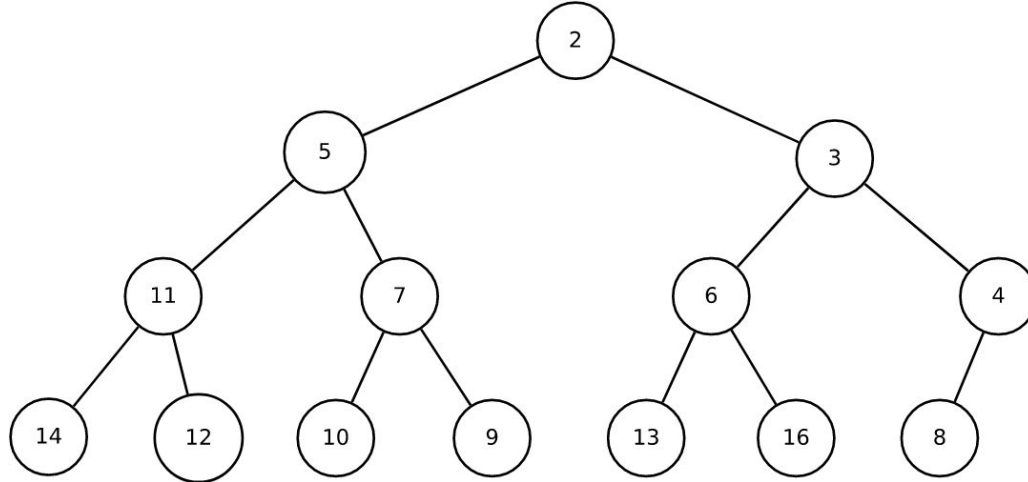
- Binomial Heap
- Fibonacci Heap
- Leftist Heap
- K-ary Heap

Implementation Approaches



Binary Heap Implementation

- In order to make our heap work efficiently, we will take advantage of the logarithmic nature of the binary tree to represent our heap. In order to guarantee logarithmic performance, we must keep our tree balanced.



Operations on Heap



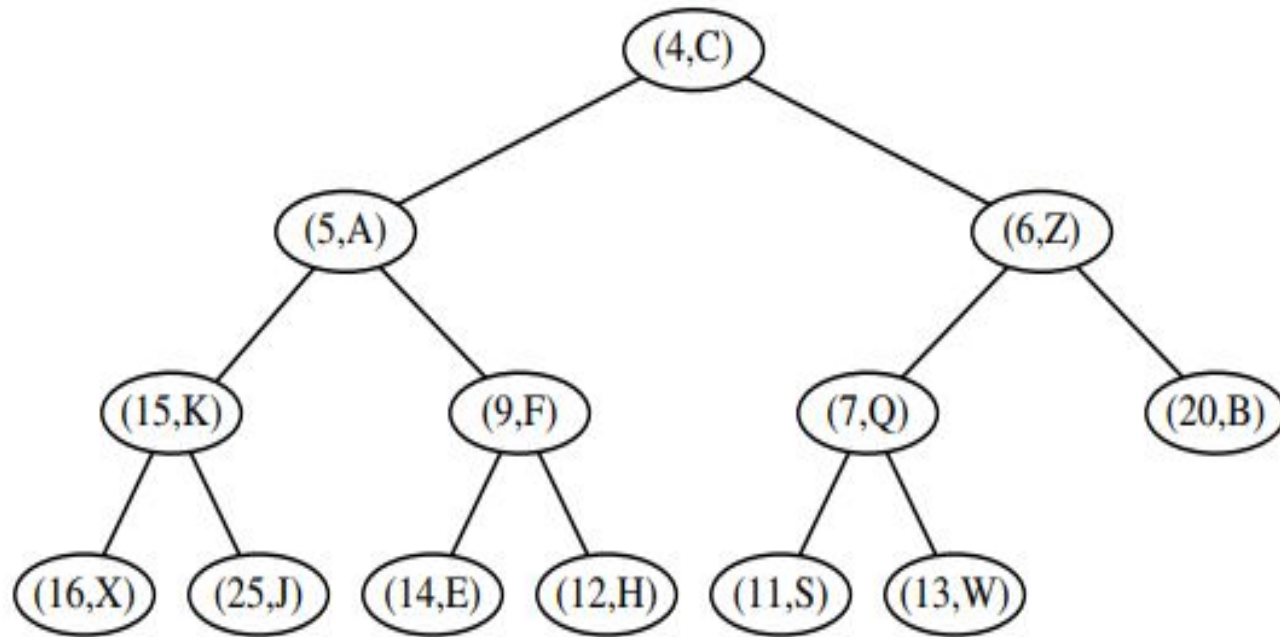
I. Adding an Item to the Heap

- We store the key value at a new node of the tree.
- To maintain the complete binary tree property, that new node should be placed at a position:
 - beyond the rightmost node at the bottom level of the tree, or
 - as the leftmost position of a new level, if the bottom level is already full (or if the heap is empty).
- After this action, the tree is complete, but it may violate the heap-order property.

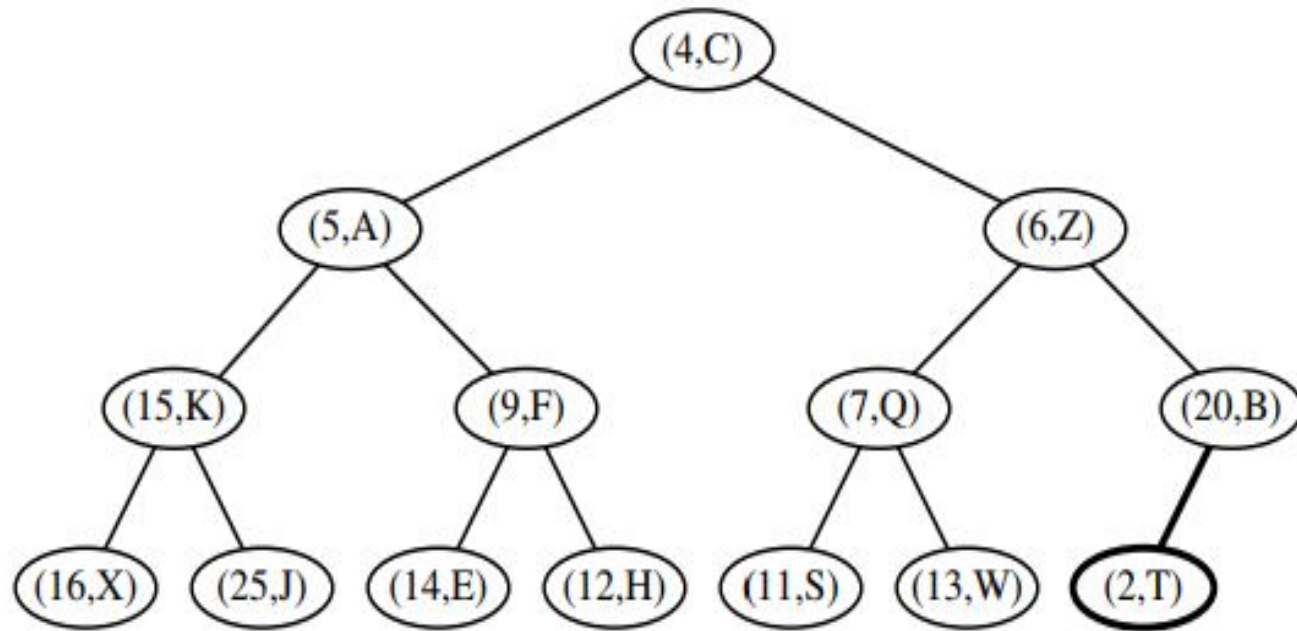
What can we do to fulfill the heap-order property ?

Up-Heap Bubbling After an Insertion

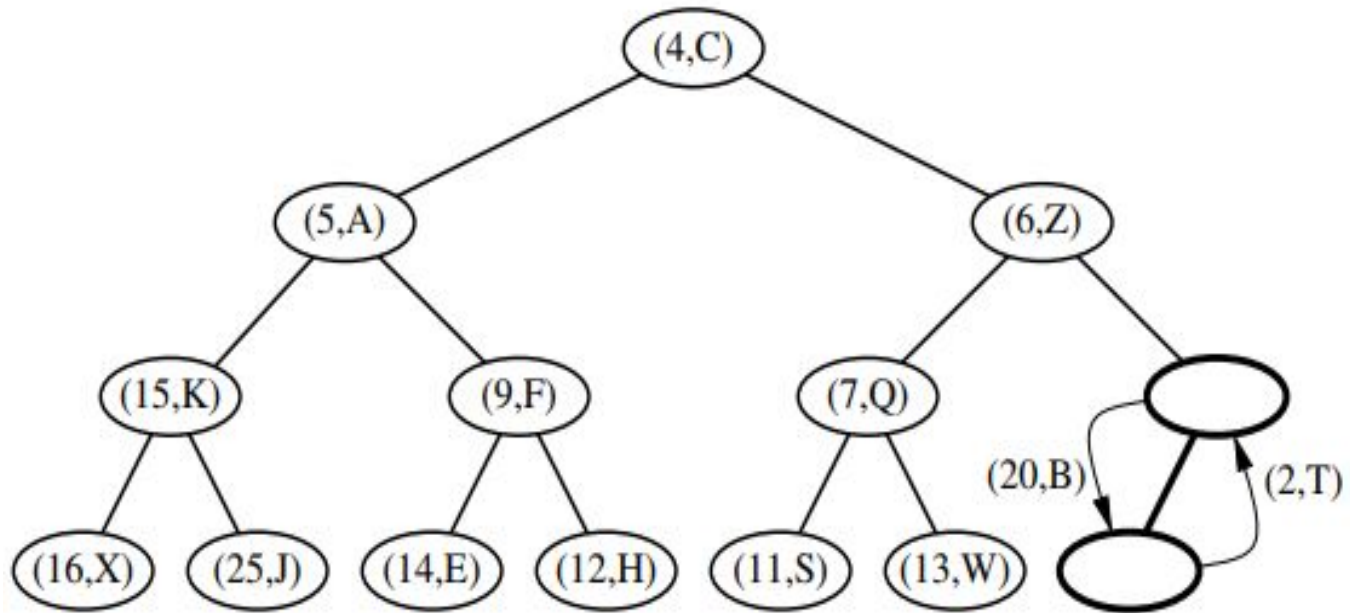
- we compare the key at position “q” to that of q’s parent “p”:
 - If key $k_q \geq k_p$, the heap-order property is satisfied and the algorithm terminates.
 - If $k_q < k_p$, swap the entries stored at positions q and p.
 - This swap causes the new item to move up one level.
 - Again the heap-order property may be violated, so we repeat the process, going up in T until no violation of the heap-order property occurs.



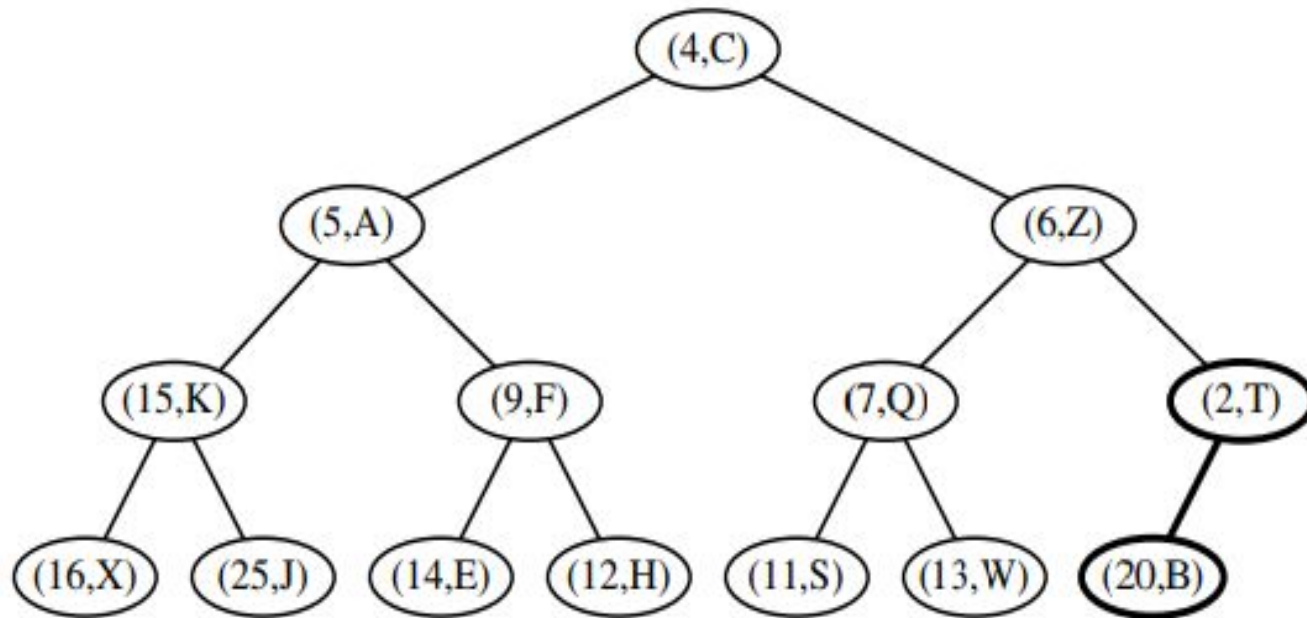
(a)



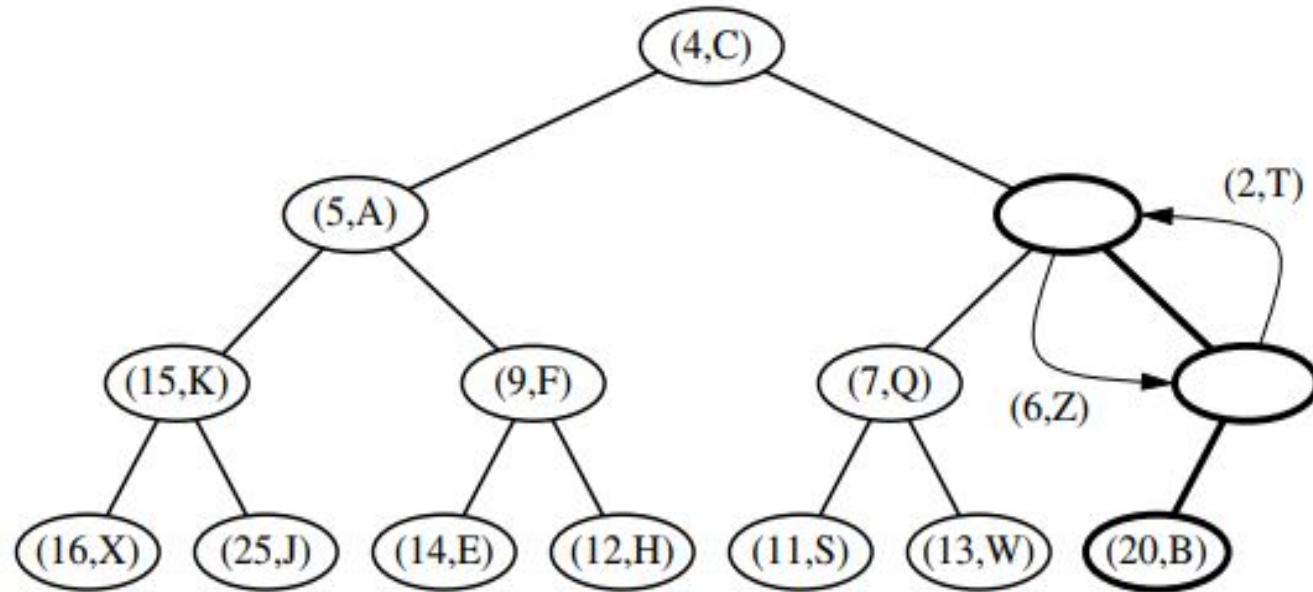
(b)



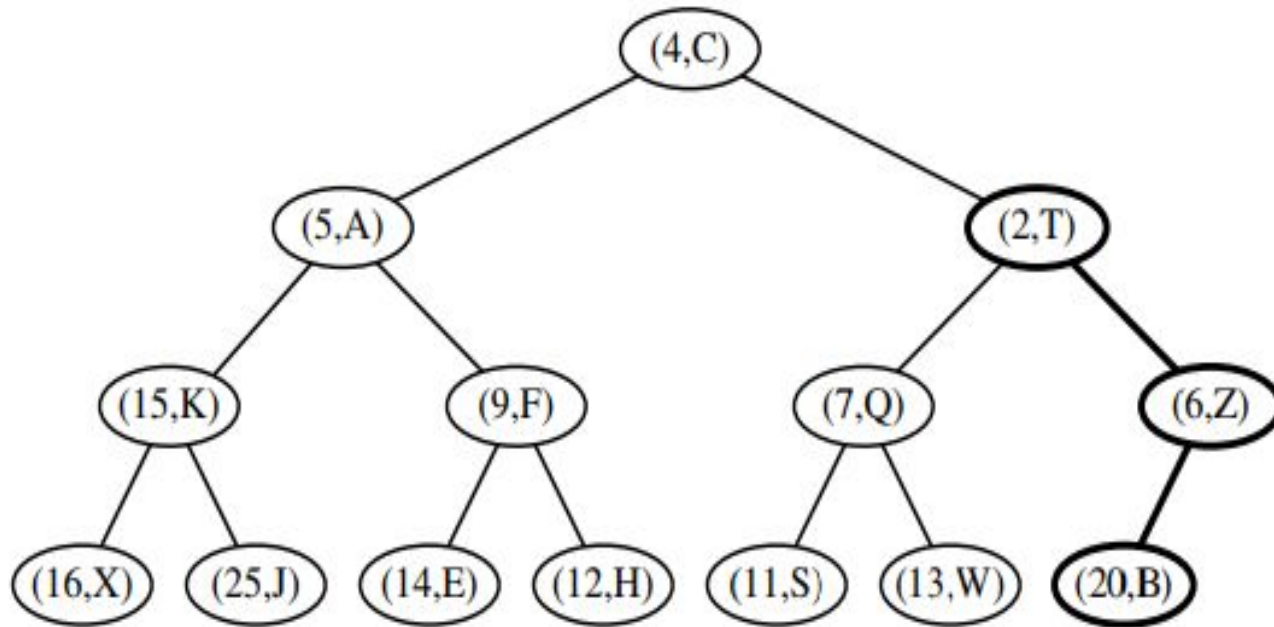
(c)



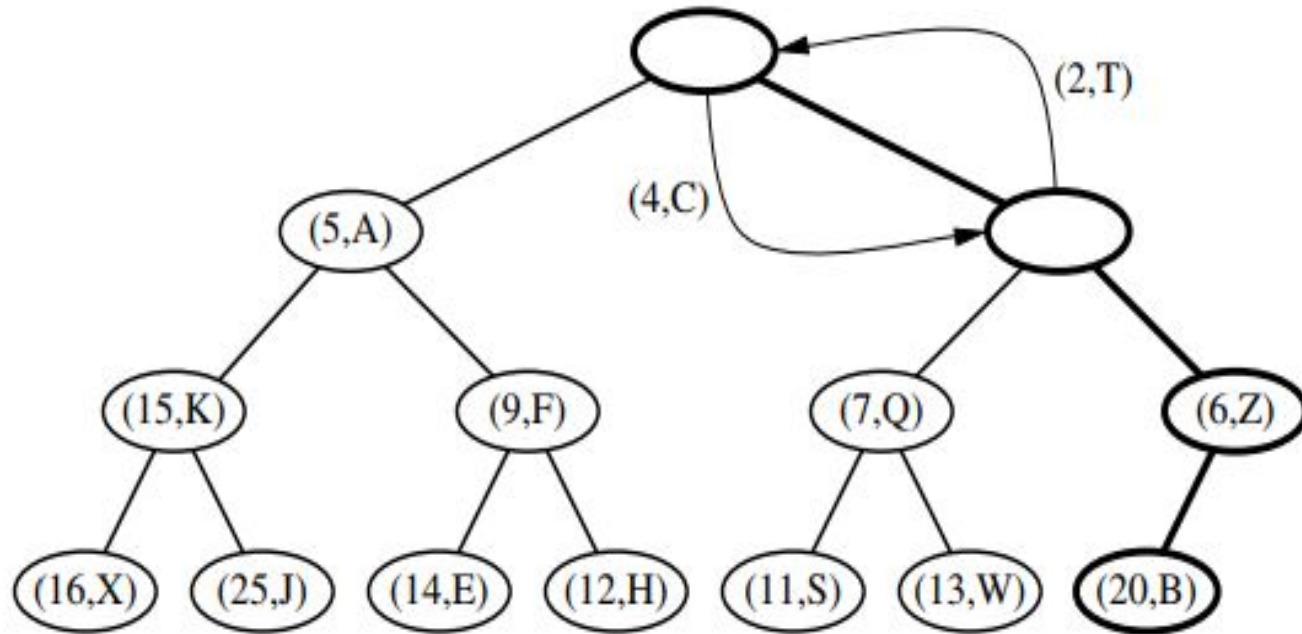
(d)



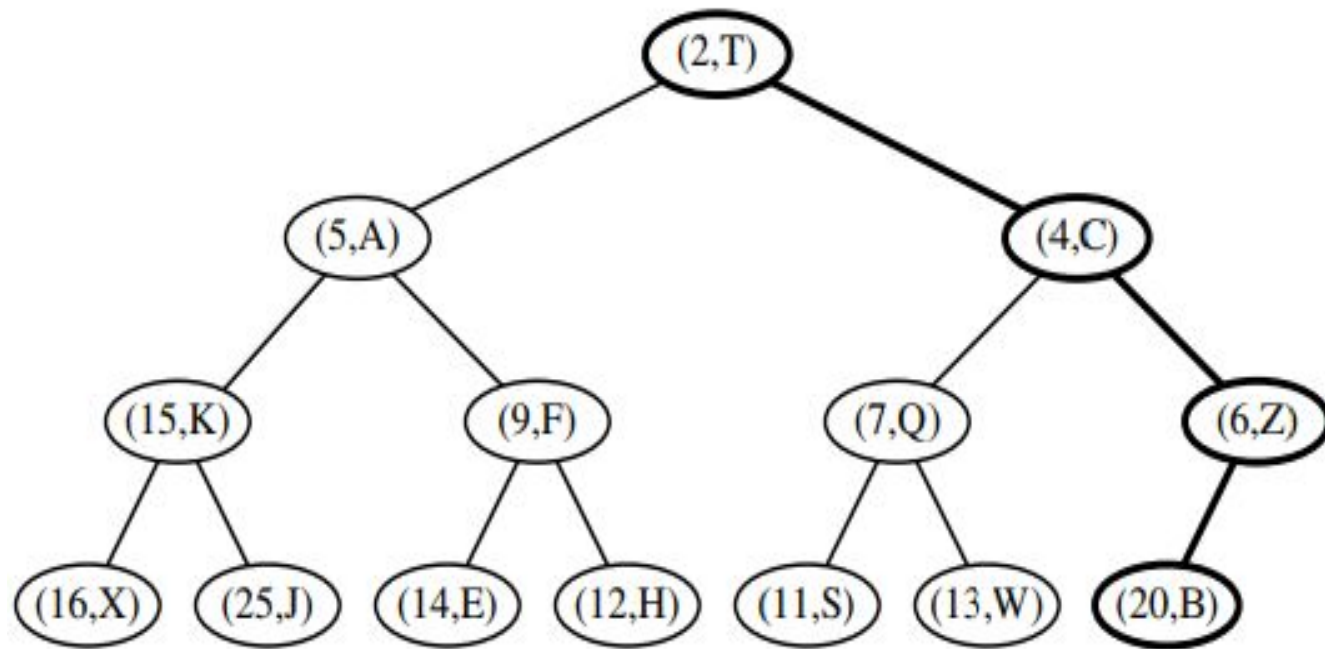
(e)



(f)



(g)



(h)

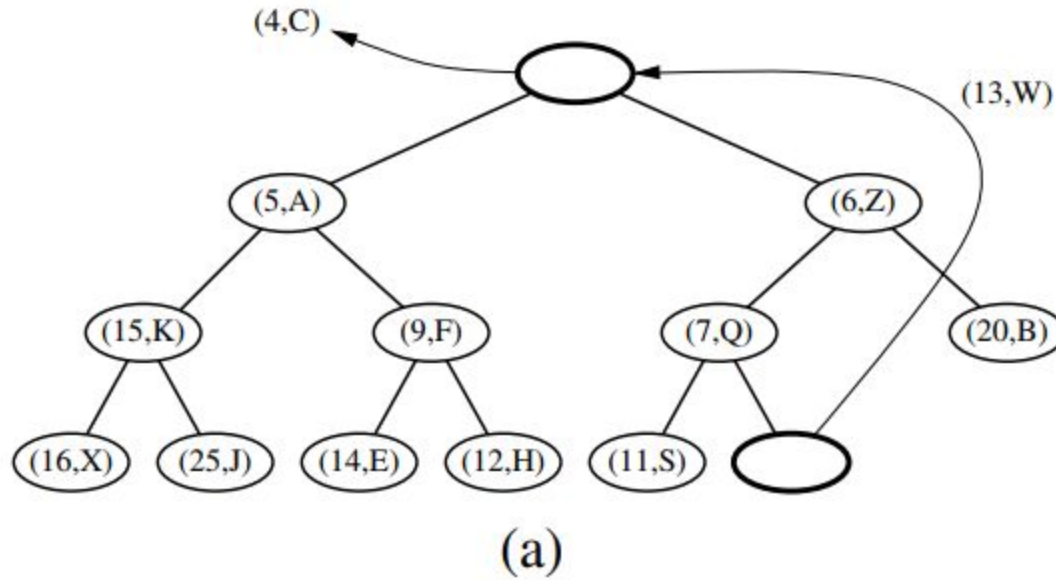
II. Removing the Item with Minimum Key

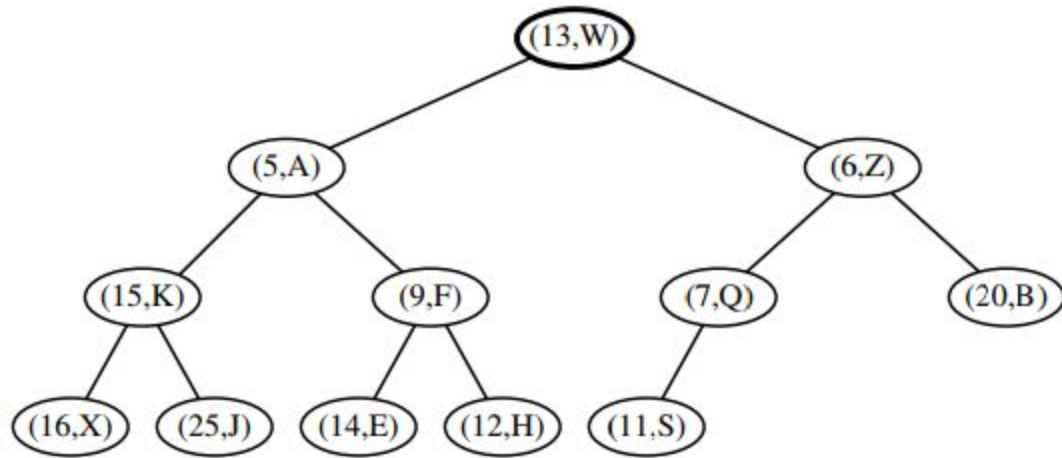
- we cannot simply delete node r , because this would leave two disconnected subtrees.
- Use the advantage of deleting the leaf at the last position q of Tree.
- To preserve the item from the last position q :
 - Swap the root r with the last position q
 - Remove the node at the last position

However, for even though T is now complete, it likely violates the heap-order property.

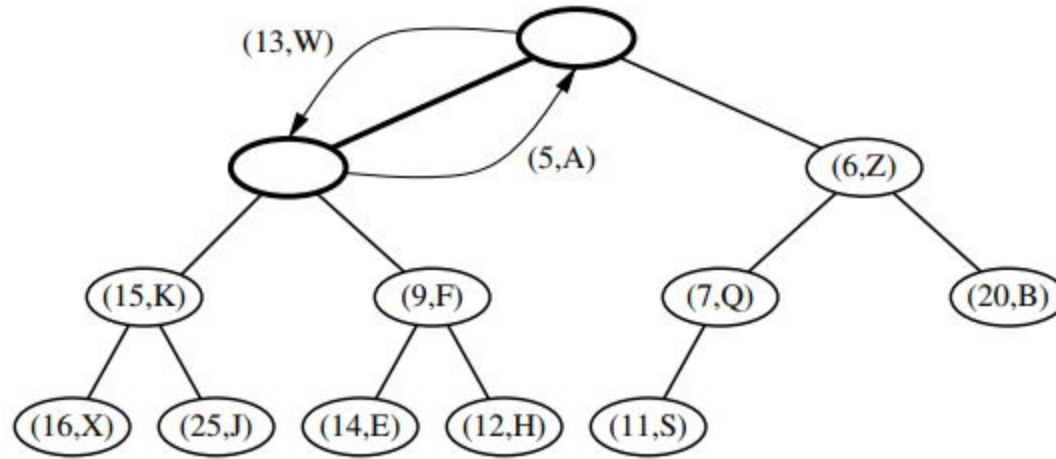
Down-Heap Bubbling After a Removal

- we distinguish two cases, where p initially denotes the root of T :
 - If p has no right child, let c be the left child of p .
 - Otherwise (p has both children), let c be a child of p with minimal key.
- If $\text{key } k_p \leq k_c$, the heap-order property is satisfied and the algorithm terminates.
- If instead $k_p > k_c$, then we need to restore the heap-order property.
 - swap the entries stored at p and c .
 - continue swapping down tree until no violation of the heap-order property occurs.

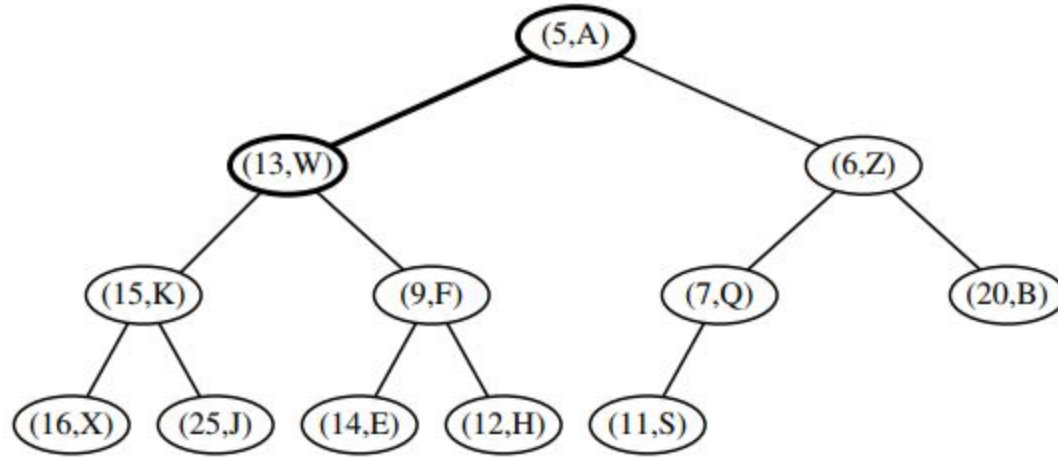




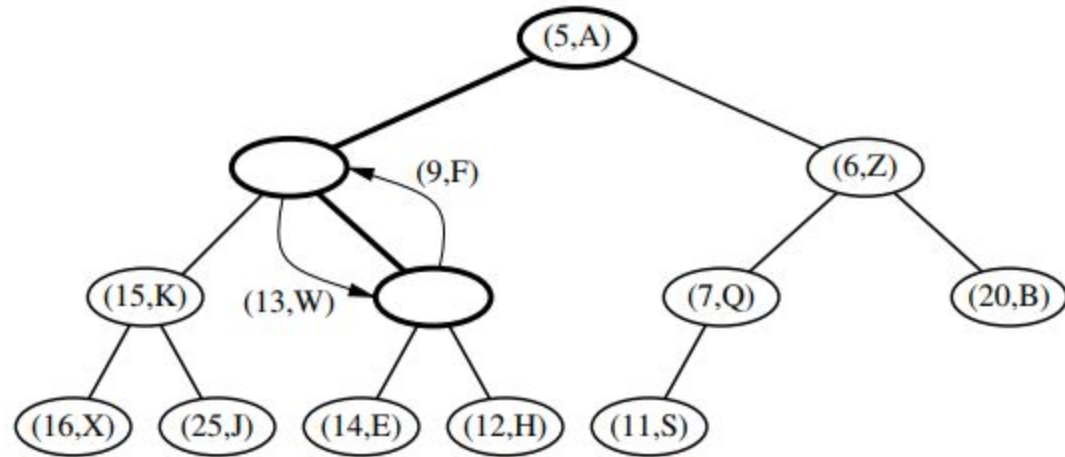
(b)



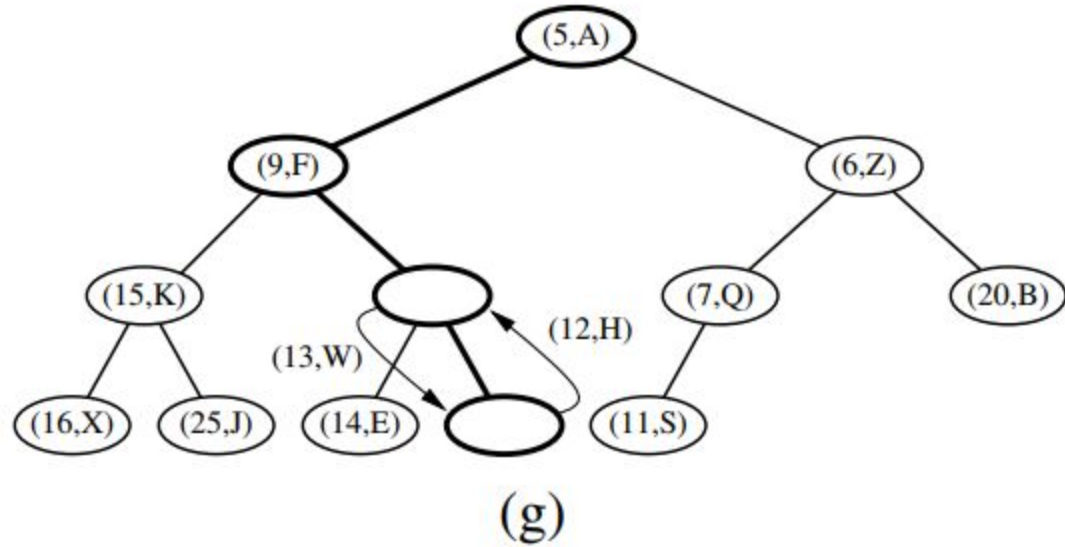
(c)

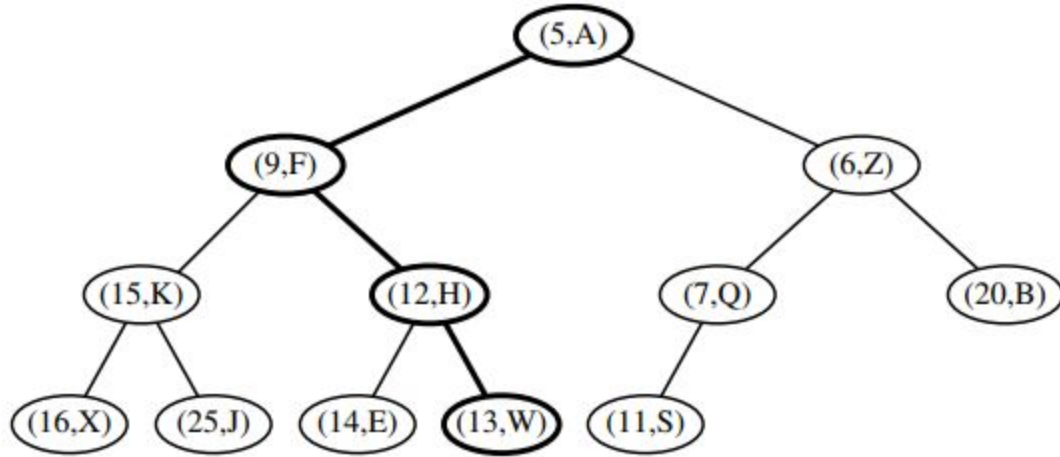


(d)



(e)



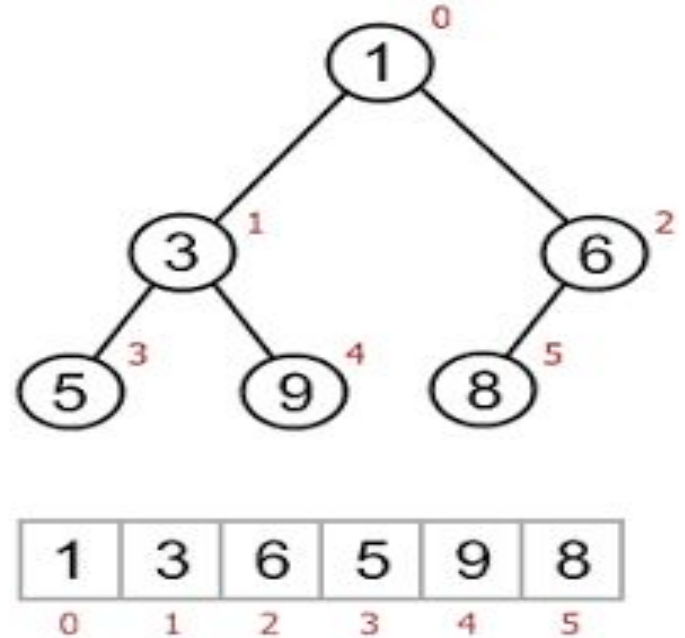


(h)

Array Representation Of Binary Heap

- Since a heap is a complete binary tree, its node values can be stored in an array, without any gaps, in a breadth-first-order.
- Let's use an implementation in which the heap elements are stored in an array starting at index 0.

- The root is $\text{array}[0]$
- The parent of $\text{array}[i]$ is $\text{array}[(i-1) // 2]$, where $i > 0$
- The left child, if any, of $\text{array}[i]$ is $\text{array}[2i + 1]$
- The right child, if any, of $\text{array}[i]$ is $\text{array}[2i + 2]$



Practice



Up-Heap Bubbling After an Insertion

Can you implement the function **heapify_up**?

```
def heappush(heap, value):  
    heap.append(value)  
    current = len(heap) - 1  
    heapify_up(current)
```

```
def heapify_up(current):  
    #write your code
```

Up-Heap Bubbling After an Insertion

To quickly test your code, run the following syntax

```
def test():
    heap = [2, 4, 5, 7, 9, 10]
    heappush(heap, 3)
    assert heap == [2, 4, 3, 7, 9, 10, 5], f"Error: expected [2, 4, 3, 7, 9, 10, 5],
but got {heap}"

    heap = [1, 2, 3, 4, 5, 6, 7, 8, 9]
    heappush(heap, 0)
    assert heap == [0, 1, 3, 4, 2, 6, 7, 8, 9, 5], f"Error: expected [0, 2, 1, 4, 5,
6, 7, 8, 9, 3], but got {heap}"

    print("Great Job !!!")

test()
```

Soulution code

```
def parent(j):  
    return (j - 1) // 2  
  
def swap(heap, i, j):  
    heap[i], heap[j] = heap[j], heap[i]  
  
def heappush(heap, value):  
    heap.append(value)  
    current = len(heap) - 1  
    heapify_up(heap, current)  
  
def heapify_up(heap, j):  
    p = parent(j)  
    if j > 0 and heap[j] < heap[p]:  
        swap(heap, j, p)  
        heapify_up(heap, p)
```

Down-Heap Bubbling After a Removal

Can you implement the function **heapify_down**?

```
def heappop(heap):  
    if not heap:  
        return None  
    min_value = heap[0]  
    heap[0] = heap[-1]  
    heap.pop()  
    current = 0  
    heapify_down(heap, len(heap), current)  
    return min_value
```

```
def heapify_down(arr, n, current):  
    #write your code
```

Down-Heap Bubbling After a Removal

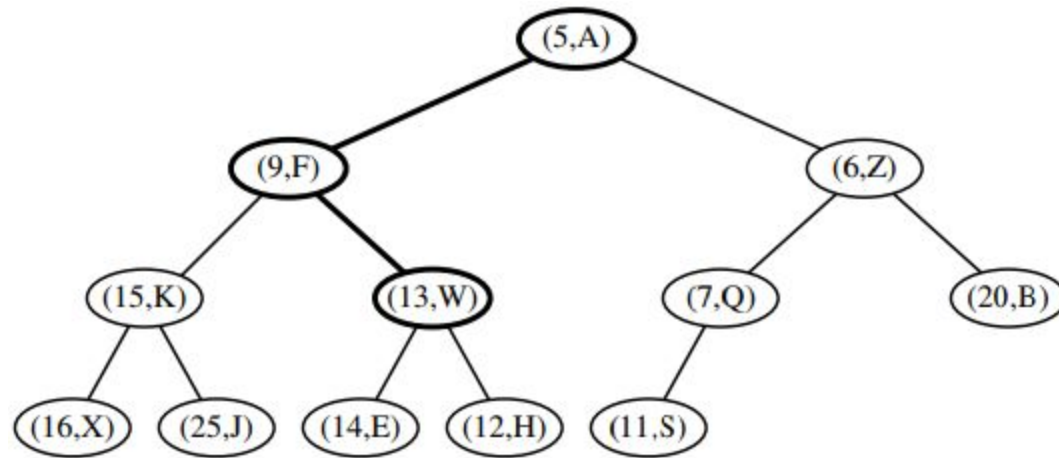
To quickly test your code, run the following syntax

```
def test():
    heap = [2, 4, 5, 7, 9, 10]
    min_val = heappop(heap)
    assert min_val == 2, f"Error: expected 2, but got {min_val}"
    assert heap == [4, 7, 5, 10, 9], f"Error: expected [4, 7, 5, 10, 9], but got {heap}"

    heap = [1, 2, 3, 4, 5, 6, 7, 8, 9]
    min_val = heappop(heap)
    assert min_val == 1, f"Error: expected 1, but got {min_val}"
    assert heap == [2, 4, 3, 8, 5, 6, 7, 9], f"Error: expected [2, 4, 3, 8, 5, 6, 7, 9], but
got {heap}"

    print("Great Job !!!")

test()
```



(f)

Soulution code

```
def swap(heap, i, j):  
    heap[i], heap[j] = heap[j], heap[i]  
  
def heapify_down(heap, n, current):  
    small_child = current  
    left = 2 * current + 1  
    right = 2 * current + 2  
  
    if left < n and heap[left] < heap[small_child]:  
        small_child = left  
  
    if right < n and heap[right] < heap[small_child]:  
        small_child = right  
  
    if small_child != current:  
        swap(heap, current, small_child)  
        heapify_down(heap, n, small_child)
```

Heapq module library



Heap sort



How to identify problems



- ★ Whenever a problem statement indicates that you're looking for some extreme element
- ★ Words that indicate a heap might be useful:
 - Largest
 - Smallest
 - Biggest
 - Smallest
 - Best
 - Worst
 - Top
 - Bottom
 - Maximum
 - Minimum
 - Optimal

Problem Patterns

- A. Top k pattern
- B. Merge K Sorted Pattern
- C. Two Heaps Pattern

A. Top K pattern

- ★ A common problem that involves finding the k largest or smallest elements from a collection of n elements.
- ★ This involves building a max (or min) heap from the input elements and then repeatedly extracting the maximum (or minimum) element k times to obtain the k largest (or smallest) elements

Practice Question Link



Sample code

```
def top_k_elements(nums, k):  
    # Initialize a heap to store the top k elements  
    heap = []  
  
    for num in nums:  
        # If the heap has less than k elements, add the number to the heap  
        if len(heap) < k:  
            heapq.heappush(heap, num)  
        # If the number is greater (or smaller) than the smallest (or largest)  
        # element in the heap, replace it with the current number  
        elif num > heap[0]: # For k smallest elements, use num < heap[0]  
            heapq.heappop(heap)  
            heapq.heappush(heap, num)  
  
    # Extract the k largest (or smallest) elements from the heap
```

B. Merge K Sorted Pattern

- ★ A common problem that involves merging k sorted arrays or lists into a single sorted array or list.
- ★ heap helps to keep track of the smallest (or largest) elements from each sorted array or list, allowing us to merge them in sorted order without having to sort the entire merged array or list at once.

Practice Question Link



Sample code

```
def merge_k_sorted_arrays(arrays):
    heap = [] # Create a min heap to store elements from k arrays
    for i in range(len(arrays)):
        if arrays[i]: # Check if the array is not empty
            # Push (element, array index, element index) tuple to heap
            heapq.heappush(heap, (arrays[i][0], i, 0))

    #Merge elements from k arrays using the heap
    merged = []
    while heap:
        num, arr_idx, ele_idx = heapq.heappop(heap) # Pop smallest element from heap
        merged.append(num) # Add popped element to merged array
        # Check if there are more elements in the same array
        if ele_idx + 1 < len(arrays[arr_idx]):
            # Push next element to heap
            heapq.heappush(heap, (arrays[arr_idx][ele_idx + 1], arr_idx, ele_idx + 1))
```

C. Two Heaps Pattern

- ★ involves using two heaps (or priority queues) to efficiently solve problems
- ★ solving problems that require:
 - quick access to the smallest and largest elements
 - maintaining a fixed-size window of elements with efficient insertion and removal operations.

Practice Question Link



Sample code

```
def __init__(self):
    #Initialize max heap for smaller elements and min heap for larger elements.
    self.max_heap = []
    self.min_heap = []

def insert(self, num):
    #Insert a number into the two heaps.
    if not self.max_heap or num <= -self.max_heap[0]:
        # Insert into max heap if it's empty or the number is smaller than max heap's root
        heapq.heappush(self.max_heap, -num)
    else:
        # Otherwise, insert into min heap
        heapq.heappush(self.min_heap, num)

    # Balance the heaps if their sizes differ by more than one element
    if len(self.max_heap) > len(self.min_heap) + 1:
        heapq.heappush(self.min_heap, -heapq.heappop(self.max_heap))
    elif len(self.min_heap) > len(self.max_heap):
        heapq.heappush(self.max_heap, -heapq.heappop(self.min_heap))
```

Sample code cont.

```
def get_median(self):  
    #Get the median of the numbers in the two heaps.  
    if len(self.max_heap) == len(self.min_heap):  
        # If the heaps have the same size, the median is the average of their roots  
        return (-self.max_heap[0] + self.min_heap[0]) / 2  
    else:  
        # Otherwise, the median is the root of the larger heap  
        return -self.max_heap[0]
```

Pitfalls



Common Pitfalls

- Popping from an empty heap

- If heap:

`heapq.heappop(heap)`

- Appending to a full heap

- If `len(heap) < capacity`:

`heapq.heappush(heap, val)`